

3.1 — Configure CLAUDE.md Files with Appropriate Hierarchy and Scoping

Concept

Claude reads `CLAUDE.md` files from multiple locations and merges them into a unified instruction set at the start of every session. Understanding the hierarchy and scoping rules is essential for writing instructions that apply to the right audience without polluting unrelated contexts.

The three-level hierarchy, outermost to innermost:

1. **User-level** — `~/ .claude/CLAUDE.md` . Instructions here apply to every project on the machine. Use this for personal coding style, preferred languages, and editor habits that never change.
2. **Project-level** — `./CLAUDE.md` at the repository root. Instructions here apply to every file in the project. Use this for project conventions, technology stack, and team-agreed patterns.
3. **Directory-level** — `CLAUDE.md` placed inside a subdirectory (e.g., `src/api/CLAUDE.md`). Instructions here apply only when Claude is working with files in that directory or its children. Use this for domain-specific patterns that would be noise elsewhere.

When instructions conflict, the more specific (inner) level wins. A directory-level instruction that says "always use `async/await` " overrides a project-level instruction that says "prefer callbacks."

The `.claude/rules/` pattern

Rather than cramming all instructions into a single large `CLAUDE.md` , you can break rules into focused files inside `.claude/rules/` and import them. This keeps each rule file under 200 lines and lets teams own specific files independently.

The `@import` syntax

Any `CLAUDE.md` can include another file using `@import` :

```
@import .claude/rules/common/coding-style.md
@import .claude/rules/common/testing.md
```

Claude resolves imports relative to the file that contains the directive, reads each referenced file, and treats its contents as if they were written inline. Imports can be chained: a rule file can itself `@import` a shared base.

Mental model for scoping decisions

Ask: "Who needs to know this?"

- Every project I ever work on → user-level.
- Everyone on this team, every time they touch this repo → project-level.
- Only the engineer working in this particular subsystem → directory-level.

Keeping instructions at the narrowest correct scope reduces cognitive load. A backend engineer working in `src/api/` should not have to read frontend-specific React patterns loaded from the project root.

Code Example

File layout

```
~/ .claude/  
  CLAUDE.md           ← user-level: personal style, always active  
  rules/  
    common/  
      coding-style.md  
      testing.md  
  
my-repo/  
  CLAUDE.md           ← project-level: team conventions  
  .claude/  
    rules/  
      api-patterns.md  
      db-patterns.md  
  src/  
    api/  
      CLAUDE.md       ← directory-level: API subsystem only  
    db/  
      CLAUDE.md       ← directory-level: database subsystem only
```

User-level `~/ .claude/CLAUDE.md`

```
# Personal Preferences  
  
@import rules/common/coding-style.md  
@import rules/common/testing.md  
  
- Prefer TypeScript over JavaScript.  
- Keep functions under 50 lines.
```

Project-level `my-repo/CLAUDE.md`

```
# My Repo
```

This is a Node.js REST API backed by PostgreSQL.

```
@import .claude/rules/api-patterns.md
```

```
@import .claude/rules/db-patterns.md
```

- Use conventional commits (feat, fix, chore...).
- All environment variables are documented in ``.env.example``.

Directory-level `my-repo/src/api/CLAUDE.md`

```
# API Module
```

- Every route handler must validate input with Zod before touching business logic.
- Return errors in the shape `{ error: { code, message } }`.
- Do not import directly from ``src/db/`` - use the repository layer.

Directory-level `my-repo/src/db/CLAUDE.md`

```
# Database Module
```

- All queries use parameterized statements - never string-interpolate user input.
- Migrations live in ``src/db/migrations/``. Never edit an existing migration file.

Common Mistakes

1. **Putting everything in the project root** — A single 600-line `CLAUDE.md` at the repo root loads for every file in the project. Instructions meant only for the database layer distract Claude when it is editing frontend components. Use directory-level files to limit scope.
2. **Using user-level files for project-specific conventions** — User-level instructions follow you across every project. Writing `use React hooks` in `~/.claude/CLAUDE.md` forces that instruction into your backend-only repositories. Keep project opinions at the project level.
3. **Circular or deep import chains** — `@import` is powerful but can create hard-to-trace dependency graphs. Keep import depth to two levels: a `CLAUDE.md` imports from `.claude/rules/`, and those rule files do not import further.
4. **Contradictory instructions at different levels** — If the project root says "use 2-space indentation" and a subdirectory says "use 4-space indentation," the directory wins — but the inconsistency signals a misconfiguration. Document which level owns formatting decisions and do not duplicate them.
5. **Forgetting that directory-level instructions are additive** — A directory-level `CLAUDE.md` does not replace the parent instructions; it extends them. All user-level and project-level rules are still active. Write directory-level files as supplements, not replacements.

References

- [Claude Code configuration — CLAUDE.md files](#)
- [Claude Code settings and rules organization](#)